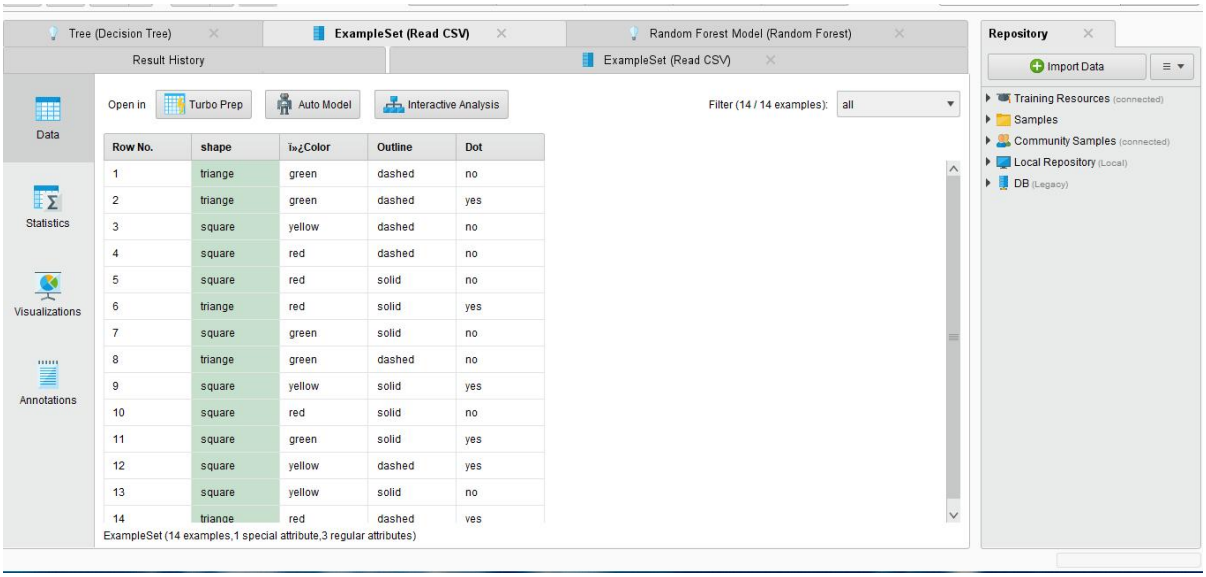
Shape Prediction using Decision Tree and Random Forest in Altair AI Studio

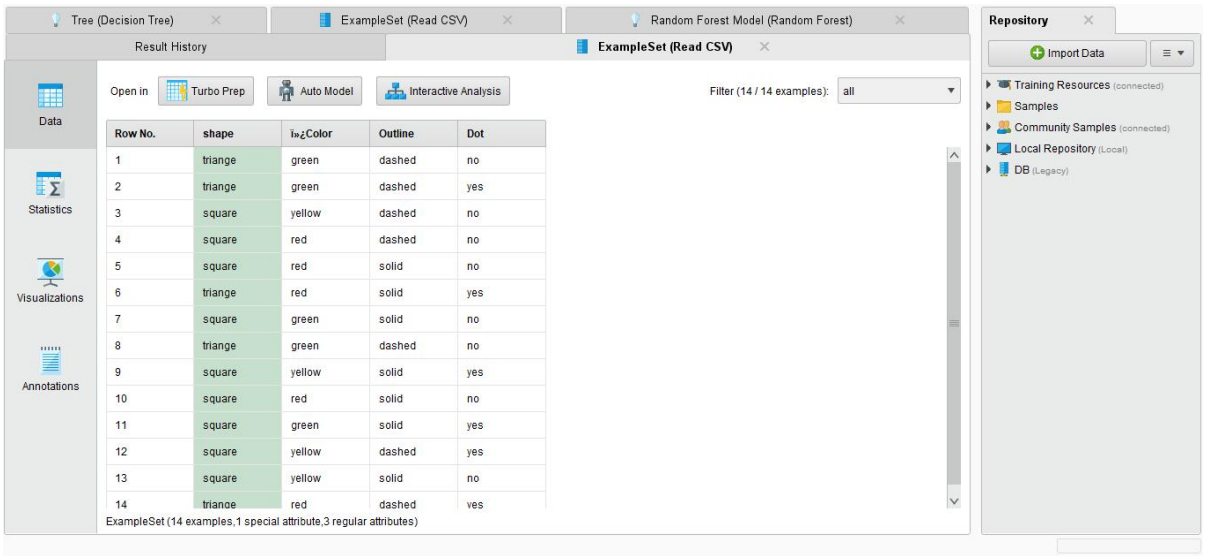
Name: Muhammad Azan	**Roll No:** 23122020

Task

The data has **14 rows** with the columns **shape**, **Color**, **Outline** and **Dot**, where the shape is known. The class work has two parts: **Part 1** builds a Decision Tree and a Random Forest from the 14 rows, and **Part 2** uses the Random Forest to predict the shape of 2 new rows whose shape is not known.

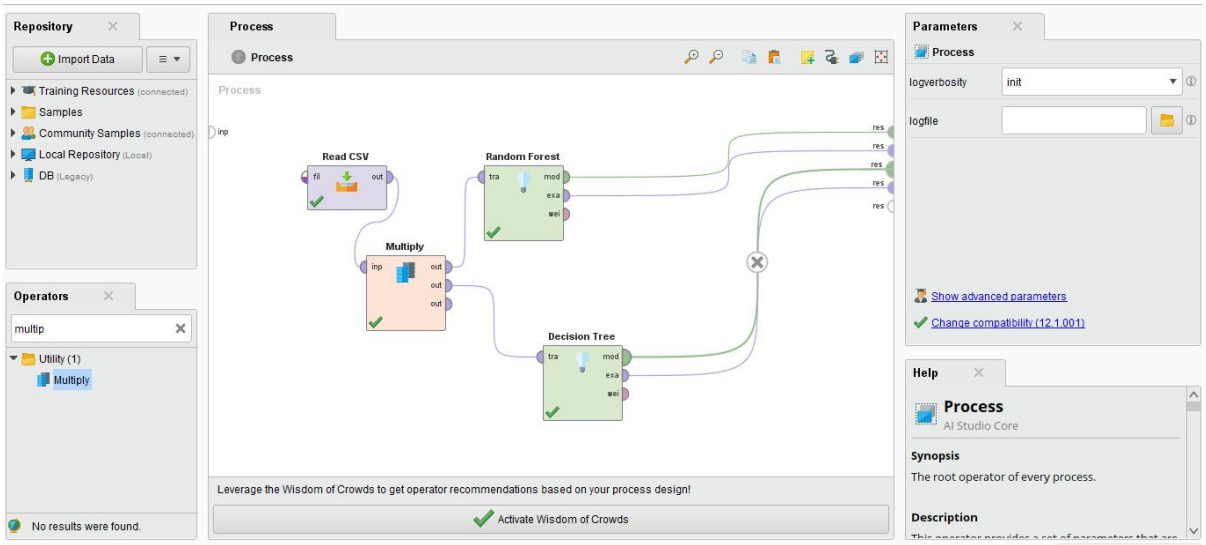


The 14 rows loaded with Read CSV. The green shape column is the label (9 square, 5 triange).



The same data opened from the Result History tab.

Part 1: Decision Tree and Random Forest



Process design for Part 1.

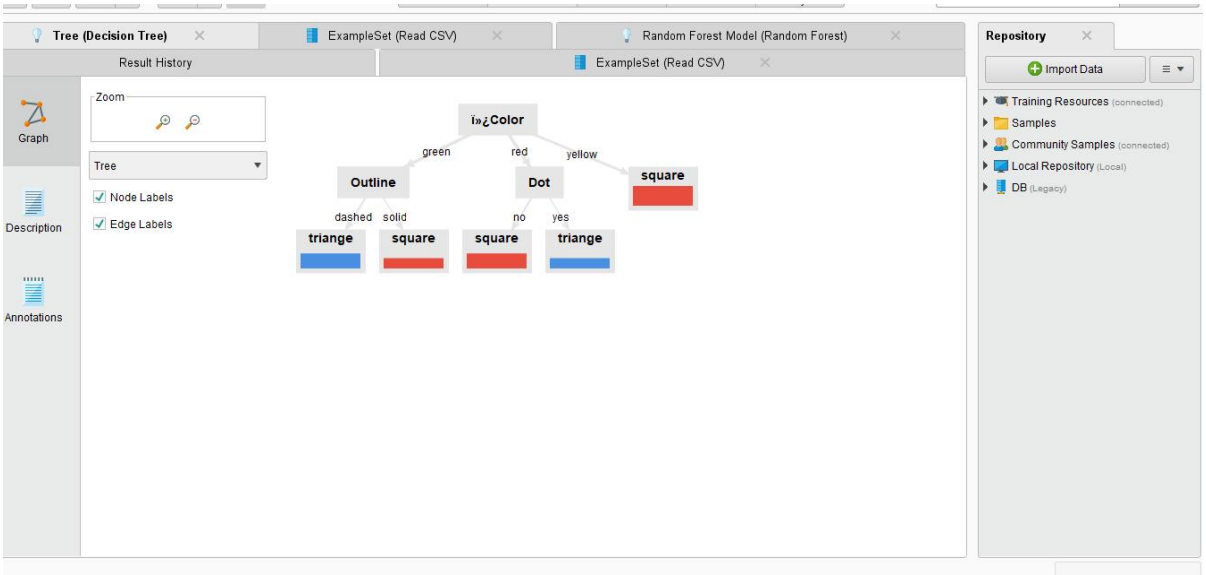
1. Read CSV: loads the 14 rows. The shape column is the label (the column to predict); Color, Outline and Dot stay as regular inputs.

2. Multiply: makes copies of the data, one for each model.

3. Decision Tree: builds a single tree from the data.

4. Random Forest: builds many trees from the same data and combines their results.

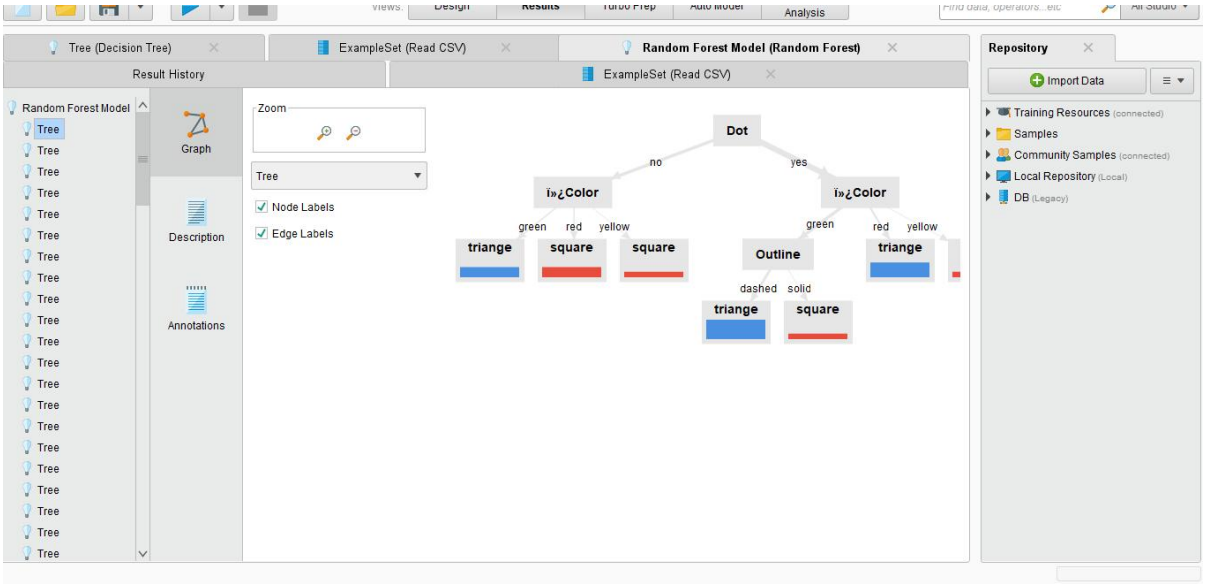
Decision Tree Result



Decision Tree (Graph view).

The tree first asks the **Color**. If it is **yellow**, the shape is a square. If it is **green**, it checks the **Outline** (dashed = triange, solid = square). If it is **red**, it checks the **Dot** (no = square, yes = triange). These rules fit all 14 rows.

Random Forest Result



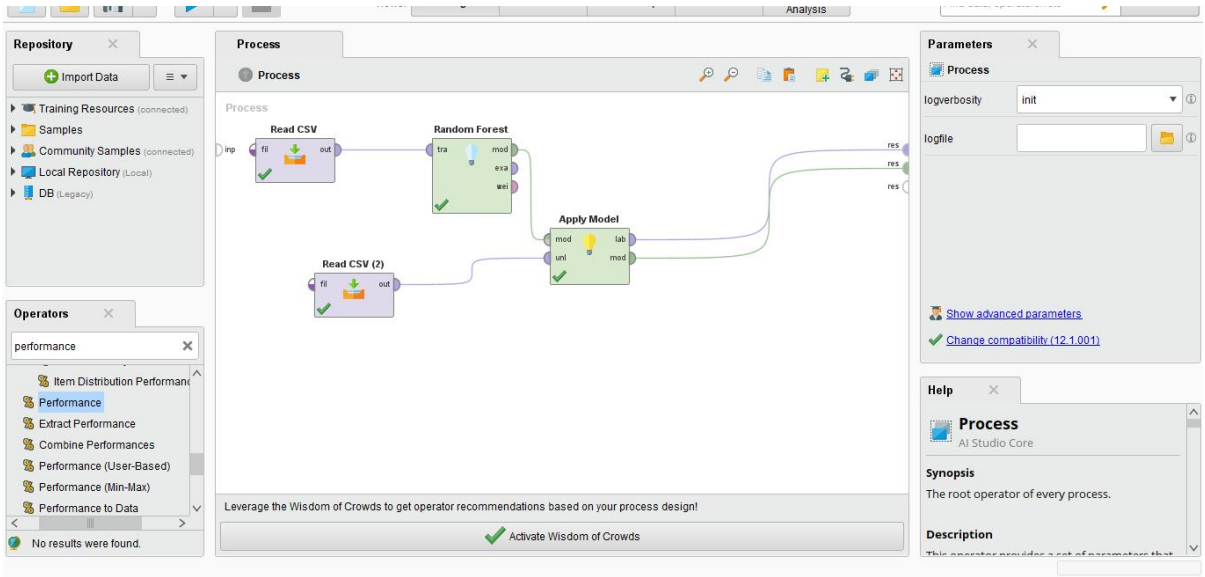
Random Forest model: the list on the left shows the individual trees, and the graph shows one selected tree.

The Random Forest builds many trees. Each tree uses a random part of the data, and the final answer is the combined vote of all trees. The tree shown starts with **Dot** and then uses **Color** and **Outline**.

Part 2: Predicting the New Shapes

Two new rows have Color, Outline and Dot, but no shape. They are stored in a second file, **prediction.csv**. The Random Forest trained on the 14 known rows is used to **predict the shape** of these 2 rows.

Color	Outline	Dot
green	solid	yes
yellow	dashed	no



Process design for Part 2.

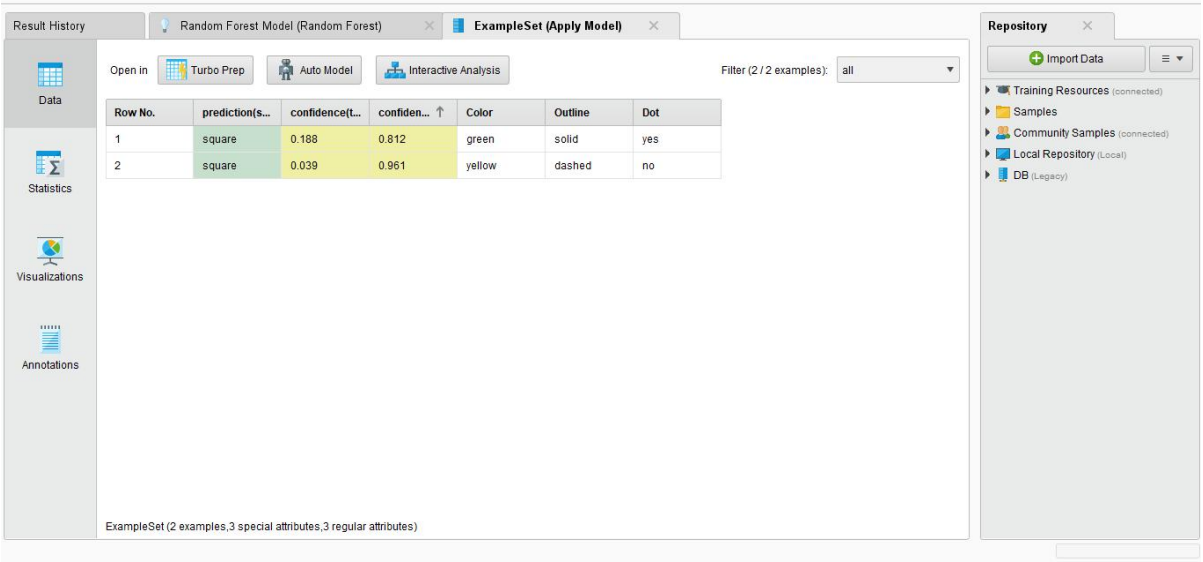
1. Read CSV: loads the 14 known rows as training data.

2. Random Forest: learns from these rows by building many decision trees.

3. Read CSV (2): loads the 2 new rows from prediction.csv (no shape column).

4. Apply Model: applies the trained Random Forest model (mod) to the 2 new rows (unl) to predict their shape. The lab output is sent to the results.

Result



Apply Model output: predicted shape for the 2 new rows.

The **prediction** column gives the answer, and the two **confidence** columns show how sure the model is for each shape (triange first, then square).

- Row 1 (green, solid, dot): predicted **square** with confidence 0.812.
- Row 2 (yellow, dashed, no dot): predicted **square** with confidence 0.961.

Conclusion: using a Random Forest trained on the 14 known rows, both new shapes were predicted to be **square**.

Model Accuracy

The screenshots do not show a Performance result, so the accuracy below is my own calculation with scikit-learn on the same 14 rows. RapidMiner's figures may differ slightly.

Model	Accuracy on training data (14 rows)	Leave-one-out cross-validation
Decision Tree	100% (14 of 14)	64.3% (9 of 14)
Random Forest (100 trees)	100% (14 of 14)	78.6% (11 of 14)

The 100% figure only shows that the models memorised the training rows. Leave-one-out cross-validation trains on 13 rows and tests on the held-out row, repeated for every row, so it is the more realistic estimate. The Random Forest (about **79%**) did better than the Decision Tree (about **64%**). With only 14 rows, each wrong row moves accuracy by about 7 points.